Customer Churn Prediction System

Software Requirements Specification (SRS)

Software Architecture and Design Description (SAD)

Software Test Plan (STP)

Prepared in accordance with IEEE Std 830-1998, IEEE Std 1016-2009 and IEEE Std 829-2008

Document identifier	HF-CCPS-SPEC
Version	2.0 (released for evaluation)
Date	31 October 2026
Prepared by	Team Hackforge
Institution	PES University, Software Engineering Mini Project
Instructor	Cvsn Reddy

Document Control

Document Information

Title	Customer Churn Prediction System: Software Requirements Specification, Software Architecture and Design Description, and Software Test Plan
Identifier	HF-CCPS-SPEC
Version	2.0
Status	Released for evaluation
Classification	Academic, public repository
Repository	github.com/Akshayshetty742/hackforge-customer-churn
Source of this document	tools/build_specification.py (regenerated from the code and the test run)

Project Team

Name	SRN	Role in this document
Akshaya B	PES1UG24AM026	Author and reviewer
Ashwin C H	PES1UG24AM900	Author and reviewer
Amar Patil	PES1UG24AM031	Author and reviewer

Revision History

Version	Date	Description
1.0	12 Oct 2026	Initial SRS, SAD and Test Plan for the command-line training pipeline (Deliverable 1).
1.1	14 Oct 2026	Added the Streamlit web application, input validation and test cases TC-14 to TC-21.
2.0	31 Oct 2026	Consolidated release after Sprints 1 to 3: batch scoring, risk explanation and what-if simulation, ML vs DML vs QML comparison, DVC and MLflow, 26 automated test cases with execution results. Restructured to the full IEEE 830, 1016 and 829 outlines.

Approvals

Role	Name	Signature	Date
Prepared by	Team Hackforge		
Reviewed by			
Approved by	Cvsn Reddy		

Table of Contents

Part A: Software Requirements Specification....5

1. Introduction....5

1.1 Purpose....5
1.2 Scope....5
1.3 Definitions, Acronyms and Abbreviations....5
1.4 References....5
1.5 Overview....6

2. Overall Description....6

2.1 Product Perspective....6
2.2 Product Functions....7
2.3 User Characteristics....8
2.4 Constraints....9
2.5 Assumptions and Dependencies....9
2.6 Apportioning of Requirements....9

3. Specific Requirements....9

3.1 External Interface Requirements....9
3.2 Functional Requirements....10
3.3 Performance Requirements....12
3.4 Logical Database Requirements....13
3.5 Design Constraints....13
3.6 Software System Attributes....14
3.7 Other Requirements....15

Part B: Software Architecture and Design Description....16

1. Introduction....16

1.1 Purpose....16
1.2 Scope....16
1.3 Design Overview....16

2. Stakeholders and Design Concerns....16

3. Design Views....16

3.1 Context Viewpoint....16
3.2 Composition Viewpoint....17
3.3 Logical Viewpoint....18
3.4 Information Viewpoint....18
3.5 Interaction Viewpoint....19
3.6 Interface Viewpoint....22
3.7 Algorithm Viewpoint....23
3.8 Deployment Viewpoint....23

4. Security Architecture....24

5. Error Handling Design....24

6. Design Rationale....24

7. Requirements Traceability....25

Part C: Software Test Plan....26

1. Introduction....26

1.1 Test Plan Identifier....26

1.2 Purpose and Scope....26
1.3 References....26
2. Test Items....26
3. Features to be Tested....26
4. Features Not to be Tested....27
5. Approach....27
5.1 Security Validation....27
5.2 Test Levels....27
5.3 Test Design Techniques....28
5.4 Automation and Continuous Integration....28
5.5 Model Evaluation Procedure....28
5.6 Test Data....28
6. Item Pass/Fail Criteria....28
7. Suspension Criteria and Resumption Requirements....28
8. Test Deliverables....28
9. Testing Tasks....29
10. Environmental Needs....29
11. Responsibilities....29
12. Staffing and Training Needs....29
13. Schedule....29
14. Risks and Contingencies....30
15. Approvals....30
16. Test Case Specifications....30
17. Requirements Traceability Matrix....35
18. Test Execution Summary....36

Part A: Software Requirements Specification

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional, interface, performance, quality and security requirements of the Customer Churn Prediction System (CCPS). It is the agreed baseline for design (Part B), verification (Part C) and acceptance. The intended audience is the development team, the course instructor and reviewers, and anyone maintaining or extending the system.

1.2 Scope

CCPS predicts whether a telecom customer is likely to cancel the service (churn), explains the main risk factors and supports retention decisions. It comprises three subsystems:

- **Training pipeline:** loads and prepares the Telco Customer Churn dataset, trains and tunes classical models, evaluates them on a held-out partition and persists the selected model with its encoders.
- **Web application:** single-customer prediction, batch scoring, model evaluation with an adjustable decision threshold, risk explanation and what-if retention simulation.
- **Paradigm benchmark:** trains classical machine learning (XGBoost), deep learning (PyTorch MLP) and quantum machine learning (PennyLane variational quantum classifier) models on an identical split, tracks every run in MLflow and serves all three predictions in the application.

CCPS is an academic decision-support system. It does not contact customers, integrate with production CRM or billing systems, or guarantee that a flagged customer will churn. All processing is local to the user's machine.

1.3 Definitions, Acronyms and Abbreviations

Table 1. Definitions and abbreviations

Term	Definition
Churn	A customer cancelling or not renewing the service; the target attribute Churn (Yes / No).
CCPS	Customer Churn Prediction System, the system specified in this document.
ML / DML / QML	Classical machine learning / deep machine learning / quantum machine learning.
XGBoost	Gradient-boosted decision tree library used for the production model.
MLP	Multi-layer perceptron, the deep learning model.
VQC	Variational quantum classifier, the quantum machine learning model.
Precision	Of the customers predicted to churn, the fraction that actually churned.
Recall	Of the customers that actually churned, the fraction predicted to churn.
ROC-AUC	Area under the receiver operating characteristic curve; ranking quality across thresholds.
Decision threshold	Churn probability at or above which a customer is classified as Churn (default 0.50).
Risk tier	Low (probability below 35%), Moderate (35% to 65%) or High (above 65%).
Held-out partition	Stratified 20% test split (random seed 42, 1,409 customers) never used for training or tuning.
DVC	Data Version Control: dataset versioning and reproducible pipeline stages.
MLflow	Experiment tracking: parameters, metrics, artifacts and models of every run.
CI	Continuous integration (GitHub Actions).
Essential / Conditional	IEEE 830 ranking: essential requirements must be met for acceptance; conditional requirements enhance the product.

1.4 References

- [1] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- [2] IEEE Std 1016-2009, IEEE Standard for Information Technology: Software Design Descriptions.

- [3] IEEE Std 829-2008, IEEE Standard for Software and System Test Documentation.
- [4] IBM Telco Customer Churn sample dataset (WA_Fn-UseC_-Telco-Customer-Churn.csv).
- **[5] Project repository:** github.com/Akshayshetty742/hackforge-customer-churn (README, DEVELOPER_GUIDE, PROJECT_REPORT, docs/reports).
- **[6] Library documentation:** scikit-learn, XGBoost, PyTorch, PennyLane, Streamlit, MLflow, DVC.

1.5 Overview

Section 2 of this part describes the product context, functions, users, constraints and assumptions. Section 3 states the specific requirements: external interfaces, functional requirements organised by feature, performance, logical data, design constraints and software system attributes, including security. Each requirement has a unique identifier, an IEEE 830 rank and the test case that verifies it (Part C).

2. Overall Description

2.1 Product Perspective

CCPS is a self-contained product that runs on a single workstation. It consumes the Telco customer dataset, produces model artifacts and experiment records, and serves predictions through a local web interface. The source code, continuous integration and backlog are hosted on GitHub.

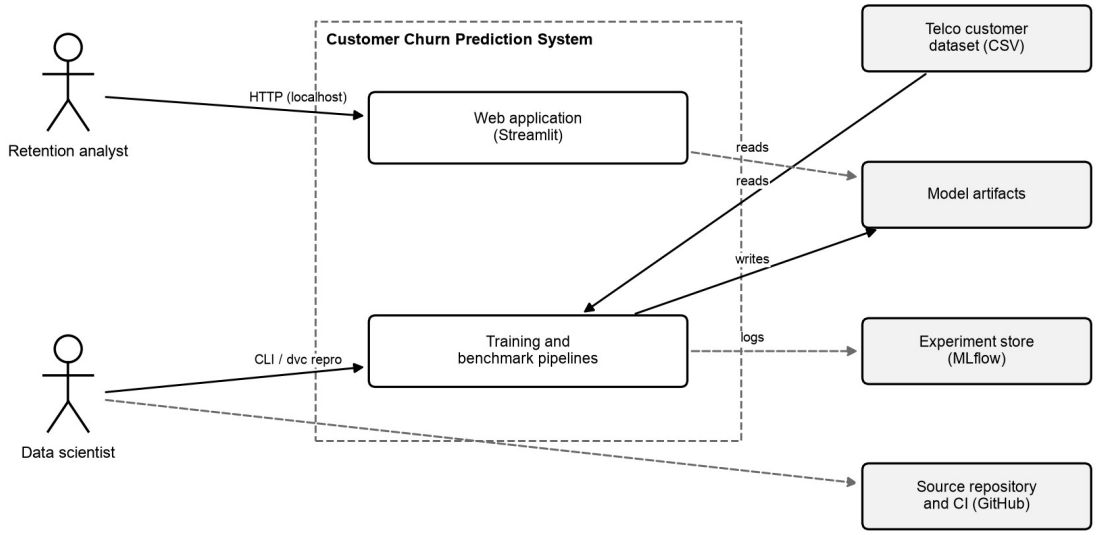


Figure 1. System context diagram

2.1.1 System Interfaces

CCPS has no runtime interface to external systems. It reads a CSV file, writes model and metric files, and stores experiment records in a local SQLite database used by MLflow.

2.1.2 User Interfaces

A browser-based interface (Streamlit) with four pages: Customer Churn Prediction, Batch Customer Scoring, Model Evaluation and About the Project. Command-line entry points are provided for training, benchmarking, testing and report generation (Part B, Section 3.6).

2.1.3 Hardware Interfaces

None beyond a standard workstation. No GPU or quantum hardware is required; the quantum model runs on a state-vector simulator.

2.1.4 Software Interfaces

Table 2. Software interfaces

Software	Purpose	Minimum version
Python	Runtime	3.10 (3.12 used in CI)
pandas, NumPy	Tabular data handling	2.0 / 1.24
scikit-learn	Preprocessing, cross-validation, metrics, estimator interface	1.4
XGBoost	Production classifier	2.0
Streamlit, Altair	Web application and charts	1.36 / 5.0
PyTorch	Deep learning model	2.2
PennyLane	Quantum circuit simulation	0.38
MLflow	Experiment tracking	3.0
DVC	Data and pipeline versioning	3.0

2.1.5 Communications Interfaces

HTTP on the local loopback interface only: port 8501 for the web application and port 5001 for the MLflow user interface (port 5000 is reserved by macOS AirPlay). No data is sent to external hosts.

2.1.6 Memory Constraints

The dataset (7,043 records) and all models fit comfortably in the memory of a standard laptop. The serialized models total about 0.3 MB.

2.1.7 Operations

- **Training mode:** batch execution of the training pipeline or the paradigm benchmark.
- **Interactive mode:** single-customer prediction, evaluation and simulation in the browser.
- **Batch mode:** scoring of a customer file through the web application.

2.1.8 Site Adaptation Requirements

All training settings (data split, hyperparameter grid, network and circuit sizes) are defined in params.yaml. Ports can be changed with the standard Streamlit and MLflow command-line options.

2.2 Product Functions

- Load, validate and prepare the customer dataset.
- Train, tune, evaluate and persist the production churn model.
- Validate a customer profile and predict churn probability, class and risk tier.
- Compare classical, deep learning and quantum model predictions for the same customer.
- Evaluate the production model on the held-out partition at a chosen decision threshold.
- Score a batch of customers and export the results.
- Explain risk drivers and simulate retention actions.
- Benchmark the three paradigms and track every experiment.

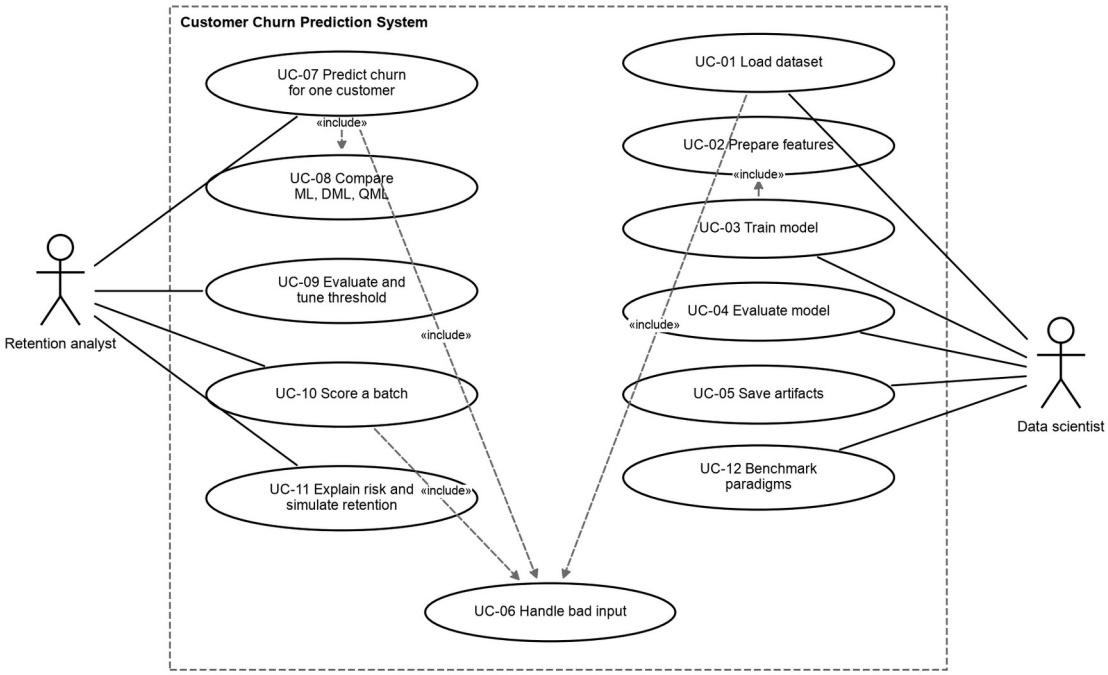


Figure 2. Use case diagram

Table 3. Use case summary

Use case	Primary actor	Description	Requirements
UC-01 Load dataset	Data scientist	Read the dataset and confirm required columns.	FR-01, FR-02
UC-02 Prepare features	Data scientist	Separate the target and encode categorical attributes.	FR-03, FR-04
UC-03 Train model	Data scientist	Train and tune the classifier with class weighting.	FR-05, FR-06, FR-10, FR-11
UC-04 Evaluate model	Data scientist	Compute metrics and the confusion matrix on held-out data.	FR-07, FR-08, FR-12
UC-05 Save artifacts	Data scientist	Persist the model with its encoders.	FR-09, FR-14
UC-06 Handle bad input	(included)	Reject missing or invalid data with a descriptive error.	FR-02, FR-16
UC-07 Predict churn	Retention analyst	Enter or load a profile and obtain a prediction.	FR-15, FR-16, FR-17
UC-08 Compare paradigms	Retention analyst	View ML, DML and QML predictions side by side.	FR-21
UC-09 Evaluate and tune	Retention analyst	Re-evaluate the model at a chosen threshold.	FR-18
UC-10 Score a batch	Retention analyst	Score a customer file and download the results.	FR-19
UC-11 Explain and simulate	Retention analyst	View risk drivers and simulate retention actions.	FR-20
UC-12 Benchmark paradigms	Data scientist	Train and compare ML, DML and QML; track runs.	FR-22, FR-23, FR-24

2.3 User Characteristics

Table 4. User classes

User class	Characteristics	Main use cases
Retention analyst	Business user; no programming knowledge; needs clear labels, explanations and plain-language results.	UC-07 to UC-11
Data scientist / ML engineer	Runs training and benchmarks from the command line; understands classification metrics.	UC-01 to UC-05, UC-12

User class	Characteristics	Main use cases
Reviewer / instructor	Inspects documentation, tests and results.	All (read-only)

2.4 Constraints

- **Data:** model quality is limited by the size and content of the public dataset (7,043 customers, 26.5% churners).
- **Imbalance:** accuracy alone is misleading; recall and precision shall always be reported (NFR-04).
- **Locality:** the system runs on one workstation and shall not transmit customer data (COM-01).
- **Quantum execution:** the quantum model runs on a classical simulator.
- **Compatibility:** a persisted model shall be used only with the encoders saved in the same run (NFR-07).

2.5 Assumptions and Dependencies

- The dataset is available at the configured path with the expected columns and Yes / No target values.
- Python and the libraries in the requirements files are installed in the active environment.
- Persisted models are loaded with library versions compatible with those used to create them.
- The deep and quantum features depend on PyTorch and PennyLane; without them the application continues to serve the classical prediction and explains which package is missing.

2.6 Apportioning of Requirements

The following capabilities are deferred to future releases: containerised deployment and a REST prediction API, production monitoring and drift detection, scheduled retraining, and execution of the quantum model on quantum hardware.

3. Specific Requirements

Requirements use "shall" for mandatory behaviour. Rank follows IEEE 830: Essential requirements must be met for acceptance; Conditional requirements enhance the product. "Verified by" lists the Part C test cases.

3.1 External Interface Requirements

3.1.1 User Interfaces

ID	Requirement	Rank	Verified by
UI-01	Every evaluation output shall label each metric by name and identify churn as the positive class.	Essential	TC-06
UI-02	The system shall provide a browser-based user interface at http://localhost:8501 with four pages: Customer Churn Prediction, Batch Customer Scoring, Model Evaluation and About the Project.	Essential	TC-14

3.1.2 Hardware Interfaces

ID	Requirement	Rank	Verified by
HW-01	The system shall run on a standard laptop or desktop CPU and shall not require a GPU.	Conditional	TC-10

3.1.3 Software Interfaces

ID	Requirement	Rank	Verified by
SW-01	The system shall read the customer dataset from a CSV file and shall report a descriptive error when the file is missing or cannot be read.	Essential	TC-01
SW-02	The system shall persist the trained model and its encoders to the configured file paths.	Essential	TC-08
SW-03	The system shall record training and benchmark runs in a local MLflow tracking store that can be browsed through the MLflow user interface on port 5001.	Conditional	TC-23

3.1.4 Communications Interfaces

ID	Requirement	Rank	Verified by
COM-01	The system shall process all customer data locally and shall not transmit customer data to any external service.	Essential	TC-11

3.2 Functional Requirements

3.2.1 Data Ingestion and Preparation

Description and stimulus/response: The data scientist starts the training pipeline. The system reads the dataset, checks the required columns and target, removes the customer identifier, separates the target and encodes categorical attributes.

ID	Requirement	Rank	Verified by
FR-01	The system shall load the Telco Customer Churn dataset from the configured CSV path.	Essential	TC-01
FR-02	If the dataset is missing, unreadable or lacks a required column, the system shall stop the operation with an error message that identifies the problem and shall not produce model, metric or report outputs.	Essential	TC-02
FR-03	The system shall separate the target attribute (Churn) from the input attributes before training.	Essential	TC-03
FR-04	The system shall encode every categorical attribute numerically with encoders that are fitted on the dataset and persisted with the model.	Essential	TC-03

3.2.2 Model Training and Tuning

Description and stimulus/response: The system trains the configured model families on the training partition, searches the hyperparameter grid with stratified cross-validation and weights the minority class.

ID	Requirement	Rank	Verified by
FR-05	The system shall train a gradient-boosted decision tree classifier (XGBoost) on the training partition.	Essential	TC-04
FR-06	The system shall select hyperparameters by exhaustive grid search with stratified k-fold cross-validation (default k = 3) using the training partition only.	Essential	TC-05
FR-10	The system shall correct class imbalance by weighting the minority class during training and shall not resample the test partition.	Essential	TC-04, TC-07
FR-11	The system shall report the best hyperparameter combination selected by the grid search.	Conditional	TC-05

3.2.3 Evaluation and Reporting

Description and stimulus/response: After training, the system scores the held-out partition once and records labelled metrics, the confusion matrix and plots.

ID	Requirement	Rank	Verified by
FR-07	The system shall compute accuracy, precision, recall, F1-score and ROC-AUC	Essential	TC-06

ID	Requirement	Rank	Verified by
	for the churn class on the held-out test partition and record them in metrics.json and the experiment store.		
FR-08	The system shall produce a confusion matrix for the held-out test partition.	Essential	TC-06
FR-12	The system shall present training configuration and evaluation results as separate, labelled outputs.	Essential	TC-06

3.2.4 Model Persistence

Description and stimulus/response: The selected model is saved with its encoders and can be reloaded to score new customers.

ID	Requirement	Rank	Verified by
FR-09	On successful completion of training, the system shall persist the selected model together with its encoders.	Essential	TC-08
FR-14	The system shall reload a persisted model with its encoders and use them to score new customer records.	Conditional	TC-09

3.2.5 Customer Profile Entry and Validation

Description and stimulus/response: The analyst fills in the form or loads a sample customer and submits it. The system validates every field before any prediction.

ID	Requirement	Rank	Verified by
FR-15	The system shall provide a customer profile form containing all 19 model inputs, grouped into Demographics, Services and Contract and Billing, each with a label and help text, and at least four one-click sample customer profiles.	Essential	TC-14
FR-16	Before predicting, the system shall validate required fields, permitted categorical values, numeric ranges and service-consistency rules (Section 3.4). If any check fails, the system shall list every error and shall not display a prediction.	Essential	TC-15

3.2.6 Churn Prediction

Description and stimulus/response: For a valid profile the system encodes the record in training column order and returns the prediction.

ID	Requirement	Rank	Verified by
FR-17	For a valid profile, the system shall display the predicted class, the churn probability as a percentage and the risk tier.	Essential	TC-16

3.2.7 Model Evaluation Interface

Description and stimulus/response: The analyst requests a live evaluation and adjusts the decision threshold to explore the precision and recall trade-off.

ID	Requirement	Rank	Verified by
FR-18	On request, the system shall re-evaluate the production model on the 1,409-customer held-out test partition and display accuracy, precision, recall, F1-score, specificity, ROC-AUC and the confusion matrix at a user-selected decision threshold between 0.05 and 0.95.	Essential	TC-17

3.2.8 Batch Scoring

Description and stimulus/response: The analyst uploads a customer file or selects the sample batch; the system scores all records and offers a download.

ID	Requirement	Rank	Verified by
FR-19	The system shall score a batch of customers (an uploaded CSV file or the 100-customer sample) in one action, return each customer's churn probability, predicted class and risk tier, and offer the result as a CSV download.	Conditional	TC-18

3.2.9 Risk Explanation and Retention Simulation

Description and stimulus/response: The system lists the factors driving a customer's risk; the analyst changes retention levers and sees the updated probability.

ID	Requirement	Rank	Verified by
FR-20	The system shall identify the main factors that raise or lower a customer's churn risk and shall let the user simulate retention actions and observe the resulting churn probability.	Conditional	TC-19

3.2.10 Multi-Paradigm Prediction and Benchmarking

Description and stimulus/response: The data scientist runs the benchmark; the analyst sees the three paradigm predictions for each customer.

ID	Requirement	Rank	Verified by
FR-21	For every valid profile, the system shall display side by side the churn probability, predicted class and risk tier produced by the classical (XGBoost), deep learning (MLP) and quantum (VQC) models using the same decision threshold.	Essential	TC-20
FR-22	The system shall provide a deep learning churn model (multi-layer perceptron) trained with class weighting and early stopping that can be persisted and reloaded with identical predictions.	Essential	TC-21
FR-23	The system shall provide a quantum machine learning churn model (variational quantum classifier, 6 qubits, 3 layers, executed on a simulator) that can be persisted and reloaded with identical predictions.	Essential	TC-22
FR-24	The system shall train the ML, DML and QML models on the same stratified split, log each run's parameters, metrics, trainable-parameter count and training time to the experiment store, and write a leaderboard (paradigm_metrics.json) and a comparison chart.	Essential	TC-23

3.2.11 Documentation

Description and stimulus/response: The project documentation supports installation, operation and evaluation.

ID	Requirement	Rank	Verified by
FR-13	The project documentation shall describe the dataset, the models, the evaluation metrics, installation and operation, and the known limitations.	Essential	TC-12

3.3 Performance Requirements

Table 5. Performance requirements

ID	Requirement	Rank	Verified by
NFR-01	A complete training run with the configured grid (Logistic Regression, Random Forest and XGBoost) shall finish in less than 300 seconds on a four-core laptop CPU without a GPU.	Essential	TC-10
NFR-11	The median time to validate, preprocess and predict one customer shall be less than 200 ms on a laptop CPU.	Essential	TC-24

3.4 Logical Database Requirements

CCPS has no database management system. Its logical data consists of the customer record, the model artifacts and the experiment records. The data dictionary below defines the customer record and the validation rules applied to it (FR-16).

Table 6. Customer record data dictionary

Attribute	Label	Type	Permitted values
gender	Gender	Categorical	Female, Male
SeniorCitizen	Senior Citizen	Integer	0 to 1 (inclusive)
Partner	Partner	Categorical	No, Yes
Dependents	Dependents	Categorical	No, Yes
tenure	Tenure (months)	Integer	0 to 120 (inclusive)
PhoneService	Phone Service	Categorical	No, Yes
MultipleLines	Multiple Lines	Categorical	No, No phone service, Yes
InternetService	Internet Service	Categorical	DSL, Fiber optic, No
OnlineSecurity	Online Security	Categorical	No, No internet service, Yes
OnlineBackup	Online Backup	Categorical	No, No internet service, Yes
DeviceProtection	Device Protection	Categorical	No, No internet service, Yes
TechSupport	Tech Support	Categorical	No, No internet service, Yes
StreamingTV	Streaming TV	Categorical	No, No internet service, Yes
StreamingMovies	Streaming Movies	Categorical	No, No internet service, Yes
Contract	Contract Type	Categorical	Month-to-month, One year, Two year
PaperlessBilling	Paperless Billing	Categorical	No, Yes
PaymentMethod	Payment Method	Categorical	Bank transfer (automatic), Credit card (automatic), Electronic check, Mailed check
MonthlyCharges	Monthly Charges ($)	Decimal	0 to 500 (inclusive)
TotalCharges	Total Charges ($)	Decimal	0 to 50000 (inclusive)

Consistency rules: total charges shall lie between 0.5 and 2 times monthly charges multiplied by tenure (and shall not exceed monthly charges when tenure is 0); customers without phone service shall have 'No phone service' for multiple lines; customers without internet service shall have 'No internet service' for every internet add-on, and customers with internet service shall not.

Table 7. Persistent data objects

Data object	Content	Retention
Production model	Dictionary {model, features_names} (customer_churn_model_xgboost.pkl)	Versioned with the code
Encoders	One label encoder per categorical attribute and the target (encoders.pkl)	Saved with the model
Paradigm models	Dictionary with model, features, paradigm, test metrics, parameter count (models/*.pkl)	Regenerated by the benchmark
Metrics	metrics.json, classification_report.json, paradigm_metrics.json	Tracked by DVC
Experiment records	MLflow SQLite store (mlflow.db) with artifacts	Local only

3.5 Design Constraints

- **Standards compliance:** documentation follows IEEE 830, 1016 and 829; Python code follows PEP 8 as checked by pylint (NFR-13).
- The production model shall remain XGBoost with class weighting; the test partition shall never be used for tuning.
- Every model shall expose the scikit-learn estimator interface (fit, predict_proba, predict).
- New data shall be encoded with the persisted encoders before scoring.

3.6 Software System Attributes

3.6.1 Reliability

ID	Requirement	Rank	Verified by
NFR-02	If required data or columns are missing, the operation shall fail with a descriptive error and shall never report fabricated metrics.	Essential	TC-02
NFR-10	If a required library or file is unavailable, the system shall stop with a visible error instead of appearing to complete.	Essential	TC-02

3.6.2 Reproducibility

ID	Requirement	Rank	Verified by
NFR-03	Given the same data, random seed and library versions, repeated runs shall produce an identical train/test split and identical predictions.	Essential	TC-13
NFR-12	Training and benchmarking shall be defined as DVC pipeline stages with all settings in params.yaml, and every run shall be recorded in MLflow with its parameters and metrics.	Essential	TC-23

3.6.3 Maintainability

ID	Requirement	Rank	Verified by
NFR-05	Data loading, preprocessing, training, evaluation and persistence shall be implemented as separate functions or modules.	Essential	TC-12
NFR-13	The code base shall achieve a pylint score of at least 8.5/10 and at least 85% combined statement and branch coverage of the application modules; both shall be enforced by continuous integration.	Essential	TC-26

3.6.4 Portability

ID	Requirement	Rank	Verified by
NFR-06	The system shall install from the published requirements files into a new Python environment and run without any developer-specific environment.	Essential	TC-10

3.6.5 Usability

ID	Requirement	Rank	Verified by
NFR-08	Every input field shall have a label and help text, and every validation message shall name the field it refers to.	Essential	TC-12, TC-15

3.6.6 Integrity and Model Quality Reporting

ID	Requirement	Rank	Verified by
NFR-04	Evaluation results shall report churn-class recall and precision alongside accuracy.	Essential	TC-06
NFR-07	The model file and the encoder file used together shall always originate from the same training run.	Essential	TC-09, TC-25

3.6.7 Security

Security objectives define what must be protected; security requirements define how.

Table 8. Security objectives

ID	Requirement	Rank	Verified by
SO-01	Confidentiality: customer data shall remain on the user's machine and personal identifiers shall not be collected or modelled unless required.	Essential	TC-11

ID	Requirement	Rank	Verified by
SO-02	Integrity: model and encoder artifacts shall not be mixed between training runs or replaced unintentionally.	Essential	TC-09, TC-25
SO-03	Safe execution: input files shall be treated strictly as data; their content shall never be executed as code.	Essential	TC-02

Table 9. Security and privacy requirements

ID	Requirement	Rank	Verified by
NFR-09	Personal identifiers (for example customer ID, name, e-mail address or telephone number) shall not be used as model inputs.	Essential	TC-11
SR-01	Data minimisation: personal identifiers and unused columns shall be removed from the model inputs.	Essential	TC-11
SR-02	Repository hygiene: credentials, private data and virtual environments shall not be committed to the public repository.	Essential	TC-11
SR-03	Artifact integrity: model and encoder files shall be regenerated and versioned together whenever the pipeline changes.	Essential	TC-09
SR-04	Local processing: the training and prediction pipelines shall not send data or predictions to any external service.	Essential	TC-11
SR-05	Input validation: missing files, missing target columns, invalid formats and invalid field values shall result in a controlled error.	Essential	TC-02, TC-15
SR-06	Experiment tracking data (MLflow database and artifacts) shall be stored on the local machine only.	Essential	TC-23
SR-07	The application shall load deep and quantum models only from the project models/ folder and shall reject any model file whose format or input features differ from the production model.	Essential	TC-20

3.7 Other Requirements

None. Requirement totals are summarised below.

Table 10. Requirement summary

Group	Requirements
Interface	7
Functional	24
Non-functional	13
Security objectives	3
Security requirements	7
Total	54

Part B: Software Architecture and Design Description

1. Introduction

1.1 Purpose

This part describes the architecture and detailed design of CCPS using the viewpoints of IEEE 1016-2009. It shows how each requirement in Part A is realised and provides the basis for implementation, maintenance and verification.

1.2 Scope

The design covers the training pipeline, the prediction service, the web application, the three model paradigms, the benchmark and the MLOps tooling (DVC, MLflow, continuous integration).

1.3 Design Overview

CCPS uses a layered architecture. A single service module (prediction_pipeline.py) holds all business rules and is shared by the web application, the training scripts and the tests. Training is organised as a pipeline whose stages are reproducible through DVC and recorded in MLflow. The deep and quantum models are adapted to the scikit-learn estimator interface, so one evaluation, logging and serving path serves all paradigms.

2. Stakeholders and Design Concerns

Table 11. Stakeholders and concerns

Stakeholder	Design concerns	Addressed in
Retention analyst	Clear inputs and results, fast response, trustworthy predictions	3.5, 3.6, NFR-11
Data scientist	Fair model comparison, reproducible experiments, configurable training	3.4, 3.5, 3.7
Maintainer	Separation of concerns, testability, code quality	3.2, 3.3, 6
Reviewer / instructor	Traceability from requirements to design and tests	7, Part C
Data owner	Privacy, local processing, artifact integrity	4

3. Design Views

3.1 Context Viewpoint

The system context is shown in Part A, Figure 1. CCPS interacts with two user classes and four external stores: the dataset, model artifacts, the experiment store and the GitHub repository with its CI service.

3.2 Composition Viewpoint

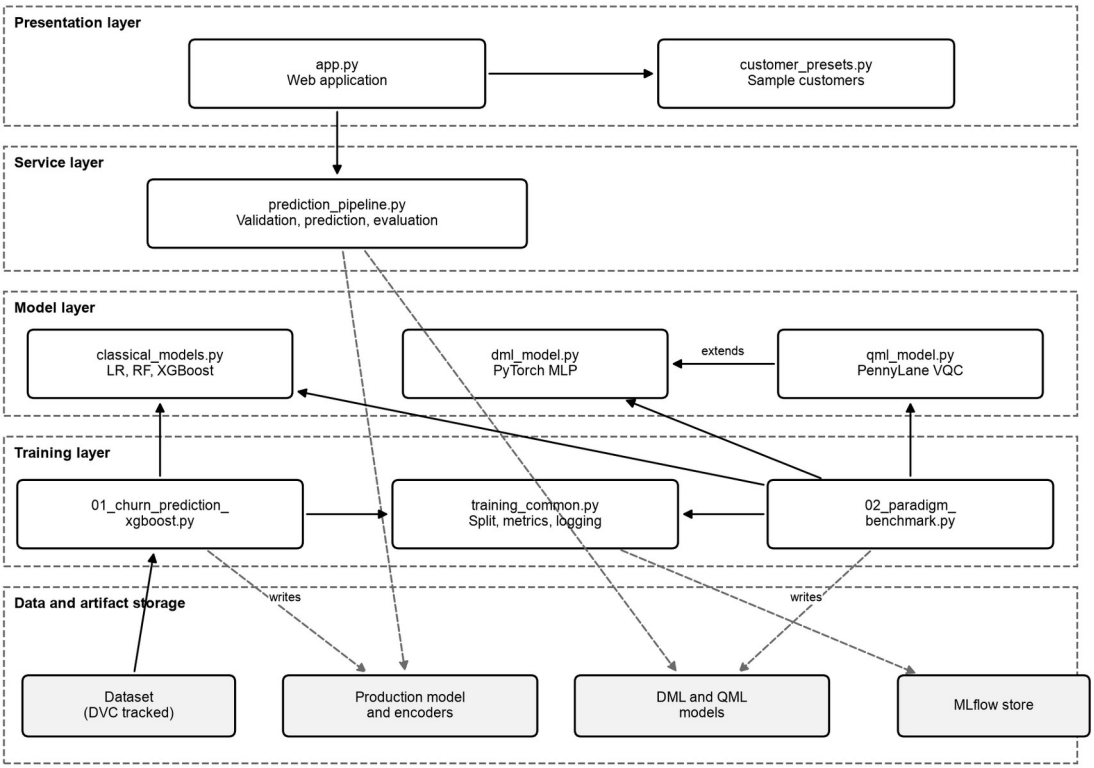


Figure 3. Component diagram

Table 12. Component responsibilities

Component	Responsibility	Provided interface
app.py	Four-page web application; layout, state and charts	Browser UI
customer_presets.py	Four realistic sample customers	get_preset_values()
prediction_pipeline.py	Validation, preprocessing, prediction, risk tiers, evaluation, batch scoring, explanation, paradigm model loading	Python API (3.6)
classical_models.py	Logistic Regression, Random Forest and XGBoost trainers with grid search	train_*()
dml_model.py	TorchTabularClassifier base and the MLP	scikit-learn estimator
qml_model.py	Variational quantum classifier	scikit-learn estimator
training_common.py	Configuration, stratified split, metrics, plots, MLflow logging	prepare_data(), evaluate_and_log_run()
01_churn_prediction_xgboost.py	Production training pipeline (DVC stage train)	CLI
02_paradigm_benchmark.py	ML vs DML vs QML benchmark (DVC stage benchmark_paradigms)	CLI

3.2.1 Architectural Patterns

Table 13. Architectural patterns

Pattern	Application	Rationale
Layered architecture	Presentation, service, model, training and storage layers	Models can change without affecting the user interface.
Pipes and filters	Training and benchmark stages; dvc.yaml	Each stage has one responsibility and defined inputs and outputs.
Adapter	TorchTabularClassifier wraps PyTorch and	All paradigms share the scikit-learn interface.

Pattern	Application	Rationale
	PennyLane models	
Template method	_build_network() overridden by the MLP and the VQC	Training, early stopping and persistence are written once.
Registry / strategy	TRAINERS and build_plan() select models from params.yaml	Model families change without code changes.
Cache	Streamlit resource cache for models	Models load once per session (NFR-11).

3.3 Logical Viewpoint

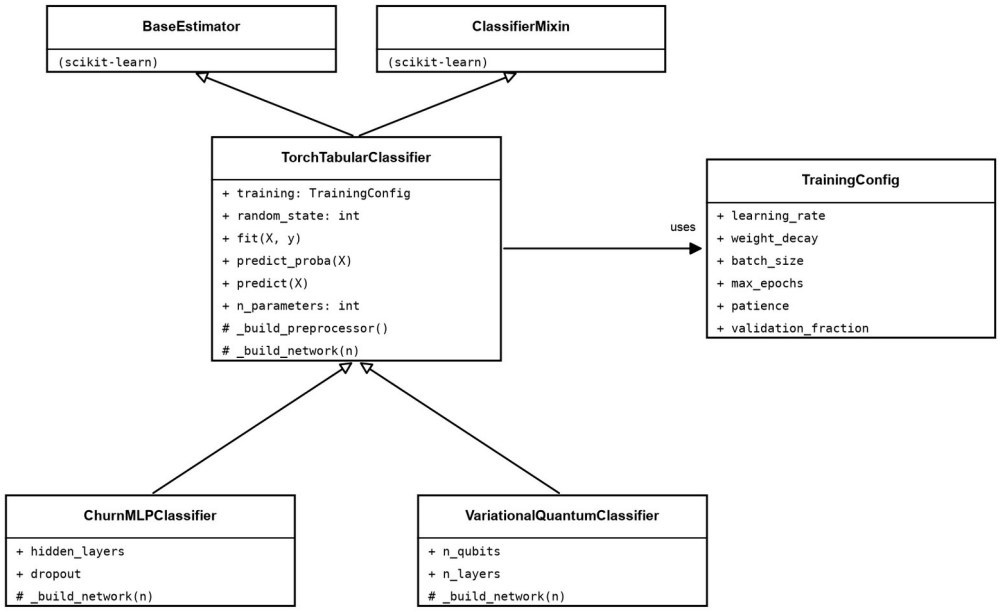


Figure 4. Class diagram of the model layer

TorchTabularClassifier implements preprocessing, class-weighted binary cross-entropy training with Adam, early stopping that restores the epoch with the lowest validation loss, prediction and pickling (weights are stored as a state dictionary and the network is rebuilt on load). Subclasses only define their network.

3.4 Information Viewpoint

The customer record and persistent data objects are defined in Part A, Section 3.4. Data flows from the raw CSV through cleaning (customer ID removed, blank total charges set to 0), label encoding and a stratified 80/20 split (seed 42). The deep and quantum models additionally apply one-hot encoding and standard scaling (45 inputs); the quantum model reduces these to 6 principal components scaled to rotation angles in [0, pi].

3.5 Interaction Viewpoint

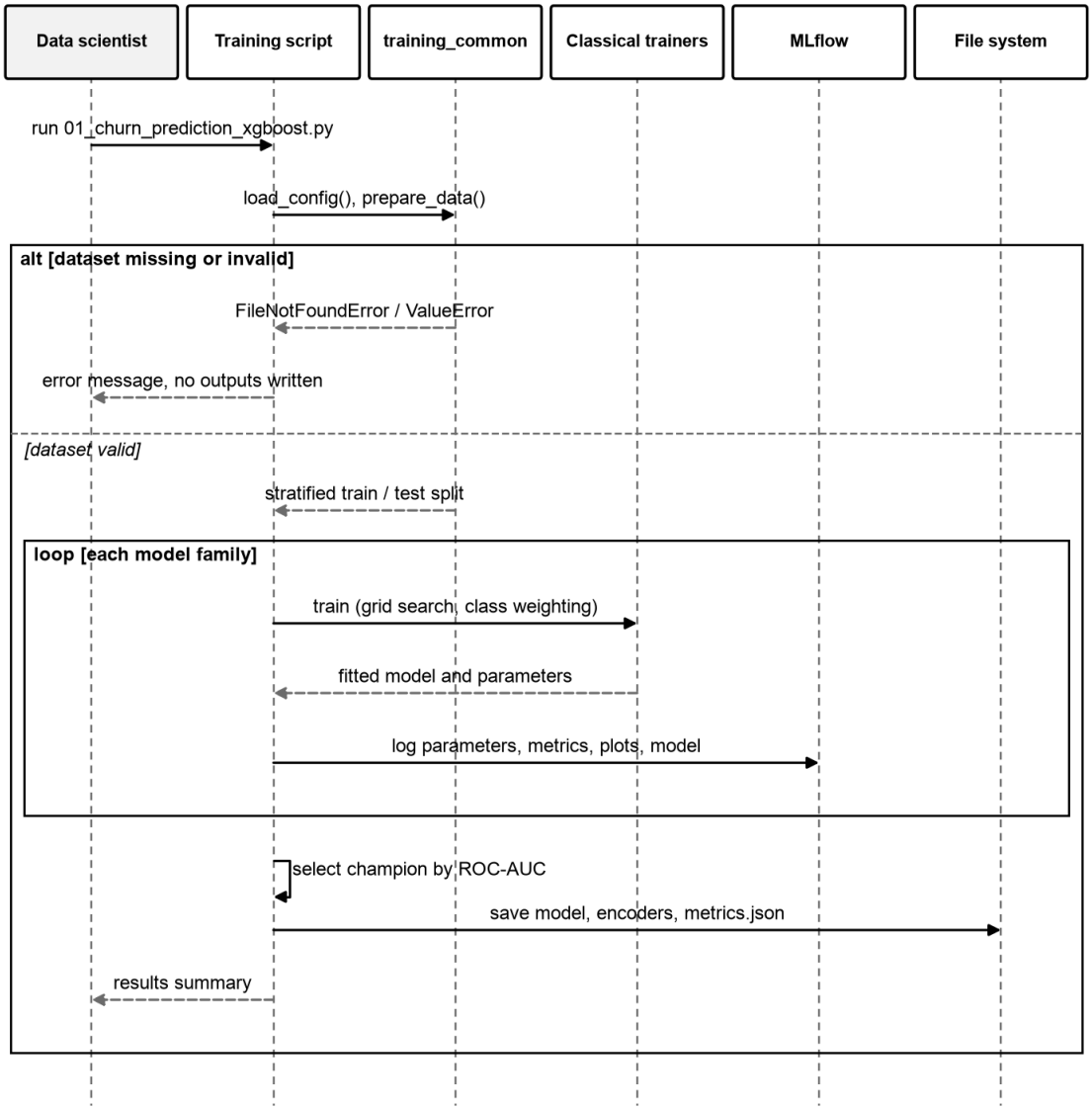


Figure 5. Sequence diagram: training, evaluation and persistence

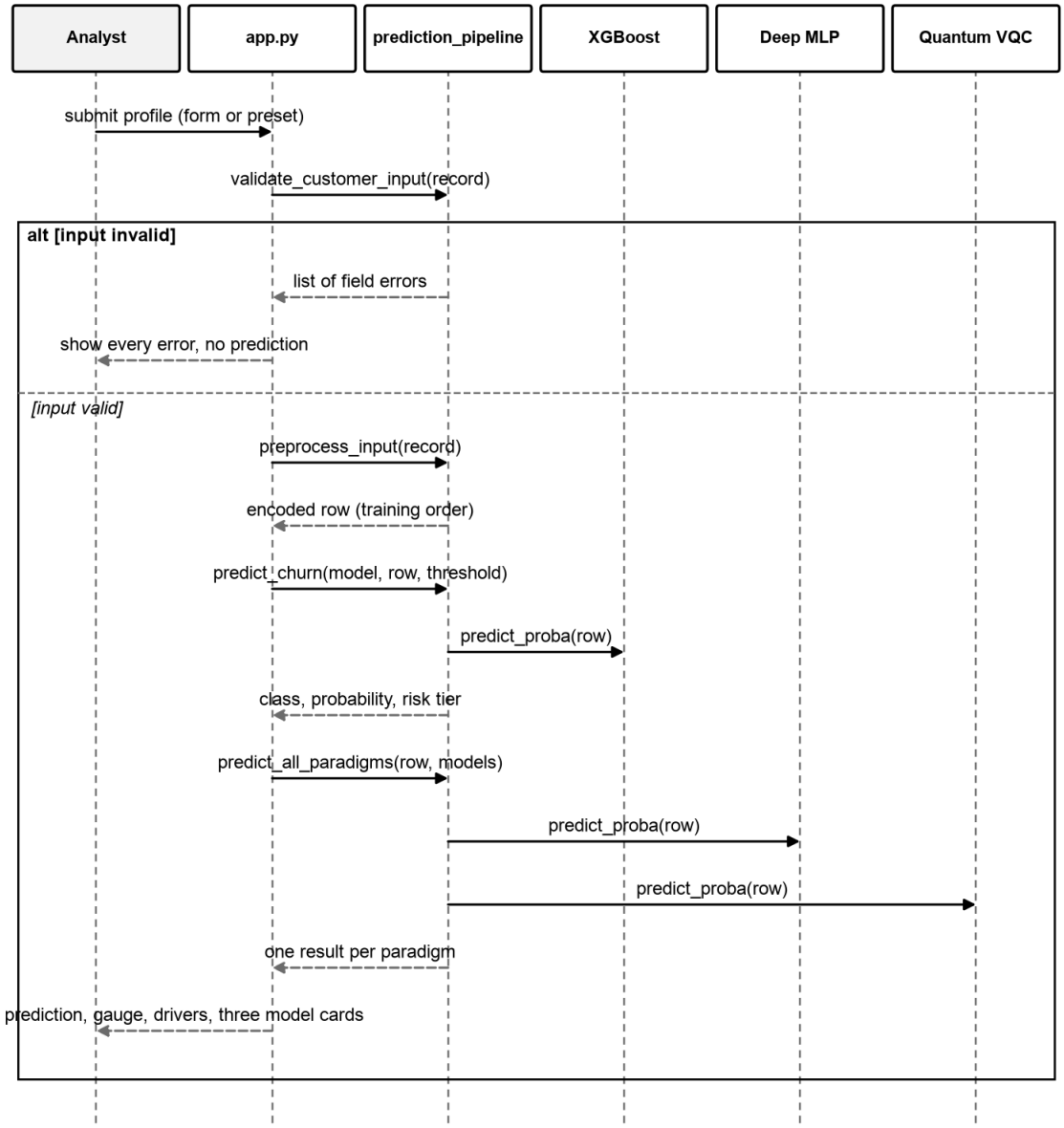


Figure 6. Sequence diagram: single-customer prediction with three paradigms

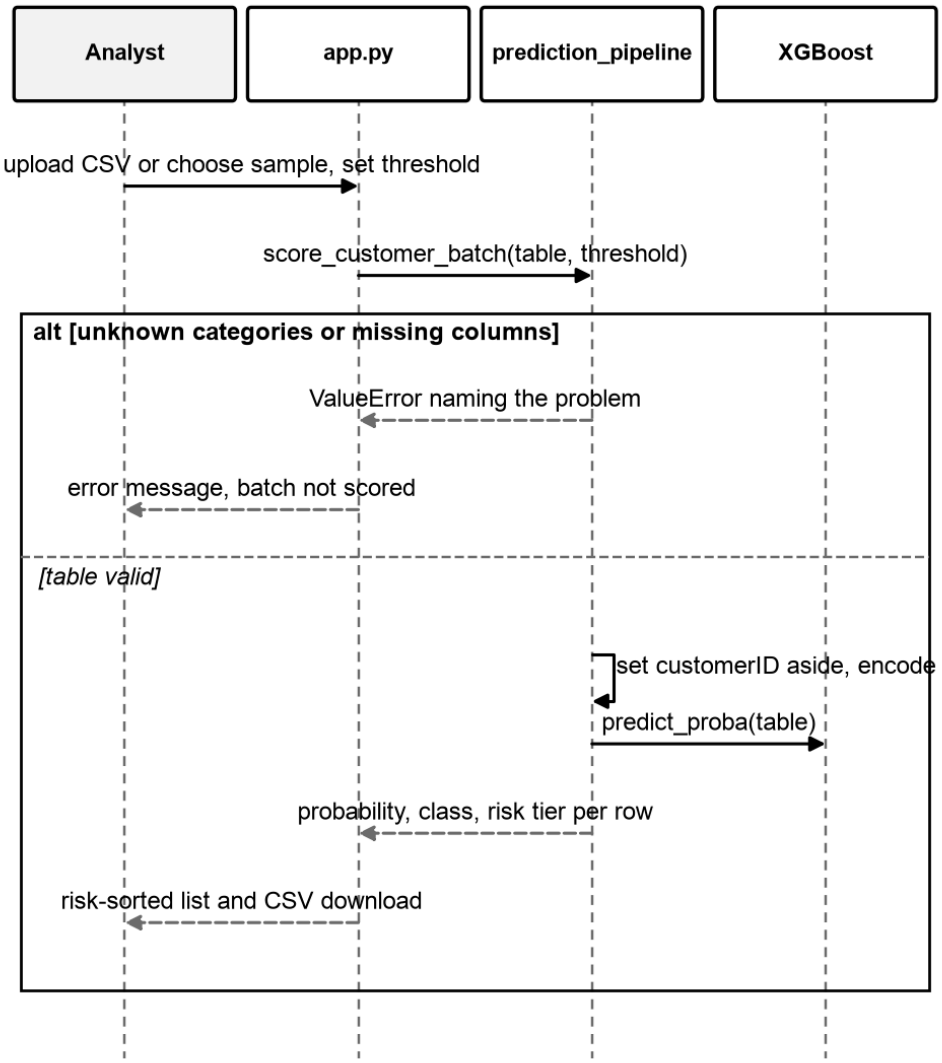


Figure 7. Sequence diagram: batch scoring

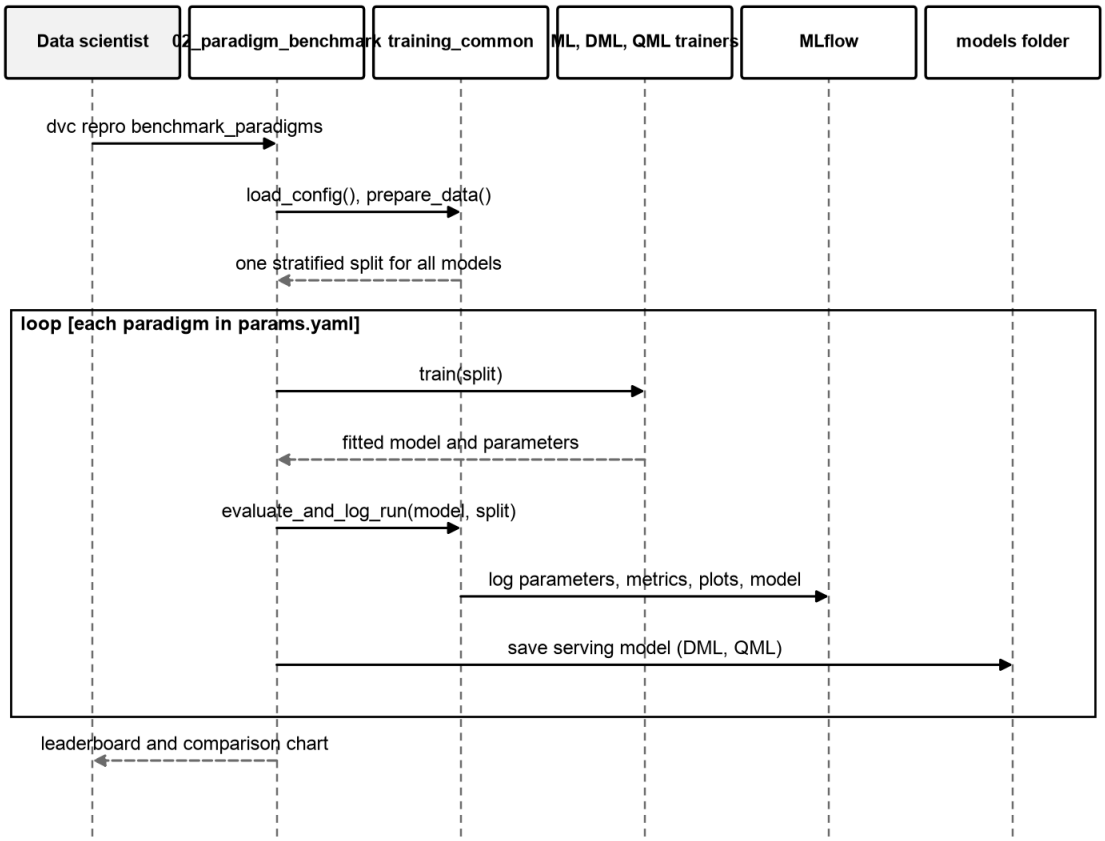


Figure 8. Sequence diagram: paradigm benchmark

3.6 Interface Viewpoint

By design CCPS exposes no network API (COM-01). Its programming interface is the prediction_pipeline module, used by the application, the training scripts and the tests.

Table 14. Prediction service API

Function	Input	Output	Errors
load_artifacts(model_path, encoder_path)	File paths (defaults: production files)	(model, feature_names, encoders)	FileNotFoundError, ValueError
validate_customer_input(record)	Dictionary of 19 fields	(is_valid, errors)	None (errors returned)
preprocess_input(record, encoders, features)	Record or table	Encoded table in training order	ValueError
predict_churn(model, row, threshold=0.5)	Encoded row	(class, probability, probabilities, tier)	None
get_risk_tier(probability)	Probability	Low, Moderate or High	None
evaluate_model_on_dataset(threshold=0.5)	Optional paths and threshold	Metrics, confusion matrix, report	FileNotFoundError, ValueError
compare_with_benchmark(results)	Live results	(all_match, comparison table)	None
score_customer_batch(table, model, encoders, features, threshold)	Customer table	Table with probability, class, tier	ValueError
explain_customer_risk(record, probability)	Record	Risk-raising and risk-lowering factors	None
load_paradigm_model(path, features)	Model file path	Validated model payload	FileNotFoundError, ImportError, ValueError
predict_all_paradigms(row, models, threshold)	Encoded row, models	One result per paradigm	None

Table 15. Command-line interface

Command	Purpose
streamlit run app.py	Start the web application (port 8501)
python 01_churn_prediction_xgboost.py (dvc repro train)	Train the classical models
python 02_paradigm_benchmark.py (dvc repro benchmark_paradigms)	Run the ML vs DML vs QML benchmark
mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5001	Browse experiments
python -m pytest	Run the automated tests
python tools/generate_test_reports.py	Run the tests and generate the test reports

3.7 Algorithm Viewpoint

Table 16. Model design

Paradigm	Model	Input processing	Structure	Training
ML	XGBoost	Label-encoded attributes	Gradient-boosted trees; depth, learning rate and number of trees from a grid search	scale_pos_weight 2.77; 3-fold stratified CV
DML	Multi-layer perceptron	One-hot and standardised (45 inputs)	64 and 32 units; Linear, BatchNorm, ReLU, Dropout 0.3	Adam (lr 0.001), class-weighted BCE, early stopping (patience 15)
QML	Variational quantum classifier	PCA to 6 features scaled to [0, pi]	6 qubits; angle embedding re-uploaded before each of 3 entangling layers; Pauli-Z readout; linear output (61 parameters)	Adam (lr 0.02), class-weighted BCE, early stopping (patience 8)

Champion selection: the training pipeline trains every configured family and exports the model with the highest value of model.champion_metric (default ROC-AUC). Imbalance correction: the positive class weight equals the ratio of non-churners to churners in the training partition (about 2.77).

3.8 Deployment Viewpoint

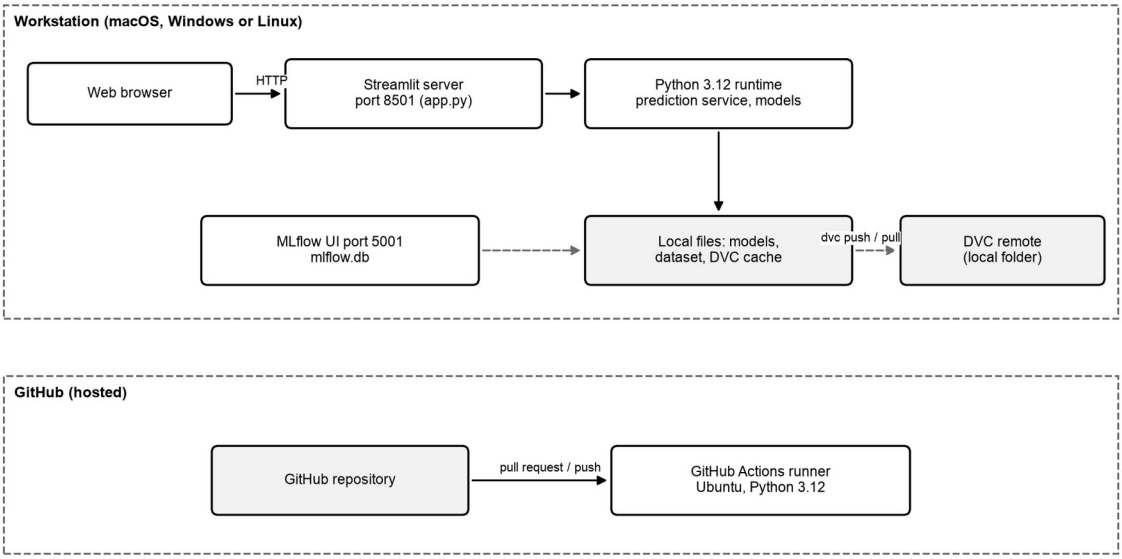


Figure 9. Deployment diagram

The application, MLflow user interface and all data run on the user's workstation. GitHub hosts the repository; GitHub Actions runs the core tests and the quality job (pylint, full test suite with coverage, report artifacts) for every pull request and every push to main.

4. Security Architecture

Trust boundaries: the user's workstation (trusted), input files supplied by users (untrusted data) and the public repository (published content).

Table 17. Threats and controls

Threat	Control	Requirements
Unexpected or malicious values in input files	Schema and value validation; files parsed as data only	SR-05, SO-03
Personal data used by the model	Identifier removed before modelling; input columns fixed by the feature list	SR-01, NFR-09
Tampered or mismatched model file	Models loaded only from models/; payload format and feature list checked before use	SR-07, NFR-07
Model and encoders from different runs	Saved together; integrity verified by tests	SR-03, SO-02
Data leaving the machine	No network clients in the code; local MLflow store	COM-01, SR-04, SR-06
Secrets committed to the repository	.gitignore rules and an automated tracked-file check	SR-02
Production model overwritten by tests	Tests redirect all outputs to temporary folders	SO-02

5. Error Handling Design

Table 18. Error handling

Condition	System behaviour	Component
Dataset missing or unreadable	Stops with a message naming the file; no outputs written	prediction_pipeline.load_dataset
Required column or target missing	Stops with a message listing the columns	load_dataset
Invalid form input	Lists every invalid field; no prediction shown	validate_customer_input
Unknown category or missing column in a batch	Error naming the problem; batch not scored	preprocess_input
params.yaml missing or invalid	Documented defaults used; warning printed	training_common.load_config
Unknown model type or paradigm in configuration	Stops listing valid options	select_trainers, build_plan
PyTorch or PennyLane not installed	Classical prediction served; card names the missing package	app.py
Paradigm model missing or incompatible	That card shows the reason; other results unaffected	load_paradigm_model

6. Design Rationale

Table 19. Design decisions

Decision	Alternatives considered	Rationale
Class weighting for imbalance	SMOTE oversampling, downsampling	SMOTE overfitted; downsampling discarded data; weighting keeps every record.
XGBoost as production model	Random Forest, Logistic Regression, MLP	Best ROC-AUC on the held-out set with built-in regularisation.
Streamlit for the interface	Flask with a custom front end	Fast delivery of an interactive, testable (AppTest) interface.
scikit-learn interface for all models	Separate code paths per paradigm	One evaluation, logging and serving path; fair comparison.
DVC and MLflow	Manual experiment records	Reproducible stages and automatic, complete run records.
Single-threaded PyTorch	Default threading	Prevents an OpenMP conflict between PyTorch and

Decision	Alternatives considered	Rationale
inference on macOS		XGBoost.
Local-only processing	Hosted service	Protects customer data; meets COM-01.

7. Requirements Traceability

Table 20. Requirements to design traceability

Requirements	Design element
FR-01 to FR-04, SW-01	training_common.prepare_data, prediction_pipeline.load_dataset
FR-05, FR-06, FR-10, FR-11	classical_models (grid search, scale_pos_weight)
FR-07, FR-08, FR-12, UI-01, NFR-04	training_common.evaluate_and_log_run, metrics.json
FR-09, FR-14, SW-02, NFR-07	01_churn_prediction_xgboost.export_champion, load_artifacts
FR-15, FR-16, UI-02, NFR-08	app.py form, customer_presets, validate_customer_input
FR-17, FR-18, FR-19, FR-20	predict_churn, evaluate_model_on_dataset, score_customer_batch, explain_customer_risk
FR-21 to FR-24, SR-07	dml_model, qml_model, 02_paradigm_benchmark, predict_all_paradigms
NFR-01, NFR-11, HW-01	Grid size in params.yaml; Streamlit resource cache
NFR-03, NFR-12, SW-03, SR-06	Fixed seeds, dvc.yaml, params.yaml, MLflow logging
NFR-05, NFR-06, NFR-13	Module structure, requirements files, pyproject.toml, CI workflow
COM-01, SR-01 to SR-05, SO-01 to SO-03, NFR-09, NFR-02, NFR-10	Security architecture (Section 4) and error handling (Section 5)

Part C: Software Test Plan

1. Introduction

1.1 Test Plan Identifier

Identifier	STP-CCPS-2.0
Items under test	CCPS release 2.0 (main branch)
Related documents	Part A (SRS), Part B (SAD), Unit Test Report, System Test Report, coverage reports

1.2 Purpose and Scope

This plan defines the scope, approach, resources and schedule of testing for CCPS, the test cases that verify every requirement in Part A, and the results of their execution.

1.3 References

IEEE Std 829-2008; Part A; Part B; test reports in docs/reports/test-reports.

2. Test Items

Table 21. Test items

Test item	Files
Prediction service	prediction_pipeline.py, customer_presets.py
Web application	app.py
Training pipeline	01_churn_prediction_xgboost.py, training_common.py, classical_models.py
Deep and quantum models	dml_model.py, qml_model.py
Paradigm benchmark	02_paradigm_benchmark.py
Configuration	params.yaml, dvc.yaml, requirements*.txt, .gitignore
Documentation	README.md, DEVELOPER_GUIDE.md, docs/

3. Features to be Tested

Table 22. Features to be tested

Test case	Feature	Requirements
TC-01	Load valid dataset	FR-01, SW-01
TC-02	Missing file / missing target handled	FR-02, NFR-02, NFR-10, SR-05, SO-03
TC-03	Target separation and encoding	FR-03, FR-04
TC-04	XGBoost training with class weighting	FR-05, FR-10
TC-05	Grid search with stratified cross validation	FR-06, FR-11
TC-06	Evaluation metrics and reports	FR-07, FR-08, FR-12, NFR-04, UI-01
TC-07	Class imbalance handling, test set untouched	FR-10
TC-08	Model and encoders saved	FR-09, SW-02
TC-09	Saved model and encoders reload as a pair	FR-14, NFR-07, SR-03, SO-02
TC-10	Fresh environment and training runtime	NFR-01, NFR-06, HW-01
TC-11	Privacy and repository hygiene	NFR-09, SR-01, SR-02, SR-04, SO-01, COM-01
TC-12	Documentation completeness	FR-13, NFR-05, NFR-08
TC-13	Repeatability	NFR-03
TC-14	Customer profile form and presets	FR-15, UI-02
TC-15	Input validation and boundary guards	FR-16, NFR-08, SR-05
TC-16	Single customer prediction with risk tier	FR-17
TC-17	Live evaluation and decision threshold tuning	FR-18
TC-18	Batch CSV scoring	FR-19

Test case	Feature	Requirements
TC-19	Churn driver explanation and what-if simulator	FR-20
TC-20	ML vs DML vs QML side-by-side prediction	FR-21, SR-07
TC-21	Deep learning model (DML)	FR-22
TC-22	Quantum machine learning model (QML)	FR-23
TC-23	Paradigm benchmark and experiment tracking	FR-24, NFR-12, SR-06, SW-03
TC-24	Prediction latency	NFR-11
TC-25	Production model and encoder integrity	NFR-07, SO-02
TC-26	Static code quality	NFR-13

4. Features Not to be Tested

Table 23. Features not to be tested

Feature	Reason
Whether a flagged customer will actually churn	Future behaviour cannot be verified in testing.
Retention campaigns, CRM integration	Outside the system scope (Part A, 1.2).
Internal correctness of third-party libraries	Vendor responsibility; correct use is tested.
Quantum hardware execution	The system targets a simulator.
Formal security certification	Outside the academic scope.

5. Approach

5.1 Security Validation

Table 24. Security validation

Check	Method	Pass criterion	Test case
Data minimisation	Inspect model inputs after preparation	No identifier (ID, name, e-mail, phone, address) is an input	TC-11
Repository hygiene	Check .gitignore and tracked files	Secrets and environments ignored; no secret files tracked	TC-11
Local processing	Scan source for network clients; inspect MLflow store	No network client; local store only	TC-11, TC-23
Input handling	Missing file, missing target, invalid values, unknown categories	Controlled error; no result reported	TC-02, TC-15
Artifact integrity	Reload model with encoders; load invalid paradigm models	Pairs load and predict; invalid files rejected	TC-09, TC-20, TC-25

5.2 Test Levels

Table 25. Test levels

Level	Scope	Tools
Unit	Validation rules, metrics, risk tiers, model classes	pytest
Integration	Preparation with training; persistence and reload; preprocessing with prediction	pytest
System	Full training run, full benchmark, web application run headless	pytest, Streamlit AppTest
Non-functional	Runtime, latency, repeatability, portability, privacy, documentation, code quality	pytest, pylint, coverage.py

5.3 Test Design Techniques

- **Equivalence partitioning:** valid and invalid classes for each categorical and numeric field.
- **Boundary value analysis:** numeric limits (for example tenure -1, 0, 120, 121) and threshold boundaries.
- **Decision tables:** phone and internet service consistency rules.
- **End-to-end scenarios:** training and benchmark pipelines executed in isolated temporary folders.
- **Analytical oracles:** a one-qubit circuit compared with its closed-form result; metrics on known inputs.

5.4 Automation and Continuous Integration

All test cases are automated with pytest and tagged with their test case identifier. GitHub Actions runs the core tests with the application requirements only, and a quality job with pylint, the full suite and statement and branch coverage, for every pull request and every push to main.

5.5 Model Evaluation Procedure

1. Use the stratified 20% held-out partition (seed 42, 1,409 customers) for every model.
2. Confirm churn is the positive class and that the partition is never used for tuning.
3. Compute accuracy, precision, recall, F1-score, ROC-AUC and the confusion matrix.
4. Compare live results of the production model with the recorded benchmark (tolerance 0.005).

5.6 Test Data

The project dataset for end-to-end tests; generated temporary CSV files for invalid-input tests; the four sample customers for prediction tests; small data slices for fast model unit tests.

6. Item Pass/Fail Criteria

- A test case passes when all of its automated tests pass.
- The release passes when every test case passes, combined statement and branch coverage of the application modules is at least 85%, and the pylint score is at least 8.5/10.
- **Severity:** Critical (privacy breach, target leakage, test data used for tuning); Major (a required function, metric or artifact fails); Minor (documentation or presentation defect).

7. Suspension Criteria and Resumption Requirements

Testing is suspended if the dataset or production model is missing, the environment cannot be installed, or a privacy or data-leakage problem is found. After correction the full suite is executed again.

8. Test Deliverables

Table 26. Test deliverables

Deliverable	Location
Software Test Plan (this part)	docs/reports/1. SRS STP SAD Report.pdf
Unit Test Report	docs/reports/5. Unit Test Report.pdf
System Test Report	docs/reports/4. System Test Report.pdf
Code and Branch Coverage Reports	docs/reports/6 and 7
Raw evidence (JUnit XML, coverage XML/JSON, pytest and pylint output)	docs/reports/test-reports/raw

9. Testing Tasks

5. Maintain test cases and their requirement links (traceability.py).
6. Automate each test case and tag it with its identifier.
7. Execute the suite locally and in CI; generate the reports.
8. Record defects, correct them and re-execute the suite.

10. Environmental Needs

Table 27. Test environment

Item	Value
Operating system	Darwin 25.3.0 (arm64)
Python	3.12.7
pandas	3.0.6
numpy	2.5.4
scikit-learn	1.9.1
xgboost	3.4.1
streamlit	1.65.0
mlflow	3.17.2
dvc	3.67.1
torch	2.15.0
pennylane	0.45.1
pytest	9.1.1
pytest-cov	7.1.0
pylint	4.1.3

11. Responsibilities

Role	Responsibility
Developer	Write and automate test cases for each backlog item; keep the suite passing.
Reviewer	Review pull requests; confirm each new feature has a linked test case.
Instructor	Review the documentation, reports and demonstration.

12. Staffing and Training Needs

Three team members. Required skills: Python, pytest, scikit-learn and Streamlit testing; training material is provided in DEVELOPER_GUIDE.md.

13. Schedule

Table 28. Test schedule

Activity	Date
Test plan v1.0 (TC-01 to TC-13)	12 October 2026
Web application test cases (TC-14 to TC-21)	14 October 2026
Sprint 1 and Sprint 2 automated tests and CI	October 2026
Test plan update and automation of all 26 test cases	30 October 2026
Testing week and report regeneration (Deliverable Part 3)	2 to 6 November 2026

14. Risks and Contingencies

Table 29. Test risks

Risk	Contingency
Library update changes model results	TC-17 and TC-25 detect it; regenerate model and encoders together.
Optional packages unavailable	Dependent tests skip; CI quality job installs all packages.
Tests overwrite the production model	All outputs redirected to temporary folders.
Documentation drifts from the code	Reports and this document are generated from the code and test run.

15. Approvals

Role	Name	Signature	Date
Test lead			
Reviewer			
Instructor	Cvsn Reddy		

16. Test Case Specifications

TC-01 Load valid dataset

Requirements	FR-01, SW-01
Level	Integration
Steps	Prepare the data with the project CSV.
Expected result	7,043 rows load; 1,409 go to the test set; all 19 feature columns are present.
Automated by	test_tc01_*
Result	Pass (1 of 1 automated tests passed)

TC-02 Missing file / missing target handled

Requirements	FR-02, NFR-02, NFR-10, SR-05, SO-03
Level	Unit
Steps	Use a missing path, a CSV without `Churn`, a bad config and a broken `params.yaml`; run the CLI with a missing file.
Expected result	A clear error names the problem; the CLI exits with "Error:" and writes no model or metrics; a broken config falls back to defaults.
Automated by	test_tc02_*, test_missing_dataset_file, test_dataset_missing_columns
Result	Pass (7 of 7 automated tests passed)

TC-03 Target separation and encoding

Requirements	FR-03, FR-04
Level	Unit
Steps	Inspect the prepared features and encoders.
Expected result	`Churn` is not an input; every input is numeric; an encoder exists for every text column.
Automated by	test_tc03_*
Result	Pass (1 of 1 automated tests passed)

TC-04 XGBoost training with class weighting

Requirements	FR-05, FR-10
Level	Integration
Steps	Train XGBoost on the training split.
Expected result	Training finishes; `scale_pos_weight` equals the class ratio (about 2.77).
Automated by	test_tc04_*
Result	Pass (1 of 1 automated tests passed)

TC-05 Grid search with stratified cross validation

Requirements	FR-06, FR-11
Level	Integration
Steps	Build and run the grid search; read the best parameters from MLflow.
Expected result	Stratified folds with shuffling are used; every grid combination is evaluated; the best value of each parameter is recorded.
Automated by	test_tc05_*
Result	Pass (3 of 3 automated tests passed)

TC-06 Evaluation metrics and reports

Requirements	FR-07, FR-08, FR-12, NFR-04, UI-01
Level	System
Steps	Run the full training pipeline.
Expected result	Accuracy, precision, recall, F1 and ROC-AUC are saved; the confusion matrix sums to 1,409; the classification report and plots are written; a run is logged for each model.
Automated by	test_tc06_*, metric tests in test_pipeline.py
Result	Pass (7 of 7 automated tests passed)

TC-07 Class imbalance handling, test set untouched

Requirements	FR-10
Level	Integration
Steps	Compare class ratios; read the logged weights.
Expected result	The test set keeps the natural churn ratio; class weighting is logged for each model.
Automated by	test_tc07_*, test_holdout_split_is_the_training_test_split
Result	Pass (3 of 3 automated tests passed)

TC-08 Model and encoders saved

Requirements	FR-09, SW-02
Level	System
Steps	Run the full pipeline.
Expected result	The model and encoder files exist and are not empty.
Automated by	test_tc08_*
Result	Pass (1 of 1 automated tests passed)

TC-09 Saved model and encoders reload as a pair

Requirements	FR-14, NFR-07, SR-03, SO-02
Level	Integration
Steps	Load the saved pair and predict for a sample customer; load with a missing file.
Expected result	The pair loads and gives a probability between 0 and 1; a missing file gives a clear error.
Automated by	test_tc09_*, test_load_artifacts, test_missing_*_file_handling
Result	Pass (4 of 4 automated tests passed)

TC-10 Fresh environment and training runtime

Requirements	NFR-01, NFR-06, HW-01
Level	Non-functional
Steps	Compare imports with the requirements files; time a full training run on CPU.
Expected result	Every third-party import is declared; the run takes under 300 s (measured 15.3 s).
Automated by	test_tc10_*, test_imports
Result	Pass (3 of 3 automated tests passed)

TC-11 Privacy and repository hygiene

Requirements	NFR-09, SR-01, SR-02, SR-04, SO-01, COM-01
Level	Non-functional
Steps	Check model inputs, `.gitignore`, tracked files and imports.
Expected result	No identifiers are inputs; secrets and environments are ignored; no secret files are tracked; no network clients are used.
Automated by	test_tc11_*, test_customer_id_is_dropped_if_present
Result	Pass (5 of 5 automated tests passed)

TC-12 Documentation completeness

Requirements	FR-13, NFR-05, NFR-08
Level	Non-functional
Steps	Check the README sections and the module docstrings.
Expected result	The README covers installation, the web app, training, model performance, limitations and tests; every module documents its purpose.
Automated by	test_tc12_*
Result	Pass (2 of 2 automated tests passed)

TC-13 Repeatability

Requirements	NFR-03
Level	Non-functional
Steps	Prepare the data twice; train twice with the same seed.
Expected result	The split and the predicted probabilities are identical.
Automated by	test_tc13_*
Result	Pass (2 of 2 automated tests passed)

TC-14 Customer profile form and presets

Requirements	FR-15, UI-02
Level	System
Steps	Start the app headless; load presets; switch pages.
Expected result	The app starts with the default preset; every field has a label and help; presets fill the form; values survive page switches.
Automated by	11 tests in test_app.py and test_pipeline.py
Result	Pass (11 of 11 automated tests passed)

TC-15 Input validation and boundary guards

Requirements	FR-16, NFR-08, SR-05
Level	Unit
Steps	Submit blank, out-of-range, wrong-type and inconsistent values; validate every dataset row.
Expected result	Each invalid case is reported with the field name and no prediction is made; all 7,043 real rows pass.
Automated by	54 tests in test_pipeline.py and test_app.py
Result	Pass (54 of 54 automated tests passed)

TC-16 Single customer prediction with risk tier

Requirements	FR-17
Level	Integration
Steps	Predict for valid customers and the four presets.
Expected result	Class, probability and risk tier are returned; each preset's risk label matches the model.
Automated by	8 tests
Result	Pass (8 of 8 automated tests passed)

TC-17 Live evaluation and decision threshold tuning

Requirements	FR-18
Level	System
Steps	Re-evaluate the model in the app and in Python at several thresholds.
Expected result	Live metrics match the saved benchmark; lowering the threshold raises recall.
Automated by	7 tests
Result	Pass (7 of 7 automated tests passed)

TC-18 Batch CSV scoring

Requirements	FR-19
Level	Integration
Steps	Score the sample batch at two thresholds.
Expected result	Every customer gets a probability, class and risk tier, sorted by risk.
Automated by	test_score_customer_batch_*
Result	Pass (2 of 2 automated tests passed)

TC-19 Churn driver explanation and what-if simulator

Requirements	FR-20
Level	Integration
Steps	Explain a high-risk customer; simulate a contract upgrade.
Expected result	Risk drivers are listed; the simulated churn probability is lower.
Automated by	test_explain_customer_risk_attribution, test_what_if_counterfactual_risk_reduction
Result	Pass (2 of 2 automated tests passed)

TC-20 ML vs DML vs QML side-by-side prediction

Requirements	FR-21, SR-07
Level	System
Steps	Predict in the app; score one customer with all models; load invalid model files.
Expected result	Three cards (ML, DML, QML) appear for valid input and none for invalid input; wrong files are rejected.
Automated by	6 tests in test_app.py and test_paradigms.py
Result	Pass (6 of 6 automated tests passed)

TC-21 Deep learning model (DML)

Requirements	FR-22
Level	Unit
Steps	Train, pickle, reload and clone the MLP.
Expected result	Probabilities are valid; reloaded predictions are identical; the best epoch is restored; the parameter count matches the architecture.
Automated by	9 tests in test_paradigms.py
Result	Pass (9 of 9 automated tests passed)

TC-22 Quantum machine learning model (QML)

Requirements	FR-23
Level	Unit
Steps	Train, pickle and reload the VQC; check a one-qubit circuit against theory.
Expected result	Probabilities are valid; reloaded predictions are identical; inputs are valid rotation angles; the circuit output equals the analytic cosine.
Automated by	10 tests in test_paradigms.py
Result	Pass (10 of 10 automated tests passed)

TC-23 Paradigm benchmark and experiment tracking

Requirements	FR-24, NFR-12, SR-06, SW-03
Level	System
Steps	Run the benchmark end to end with a local MLflow store; run it with a bad config.
Expected result	One run per model tagged ML, DML or QML; leaderboard sorted by ROC-AUC; serving models reload; bad config stops with a message.
Automated by	test_tc23_* and 3 tests in test_paradigms.py
Result	Pass (6 of 6 automated tests passed)

TC-24 Prediction latency

Requirements	NFR-11
Level	Non-functional
Steps	Time 30 single predictions after a warm-up.
Expected result	Median under 200 ms (measured 11.3 ms).
Automated by	test_tc24_*
Result	Pass (1 of 1 automated tests passed)

TC-25 Production model and encoder integrity

Requirements	NFR-07, SO-02
Level	Integration
Steps	Load the production model and encoders.
Expected result	Feature order matches training; encoders cover every text column and match the allowed values.
Automated by	test_tc25_*, test_valid_categories_match_fitted_encoders
Result	Pass (3 of 3 automated tests passed)

TC-26 Static code quality

Requirements	NFR-13
Level	Non-functional
Steps	Run pylint and coverage.
Expected result	pylint at least 8.5/10 (measured 9.82); coverage at least 85%.
Automated by	CI quality job, tools/generate_test_reports.py
Result	Pass (pylint score 9.82/10)

17. Requirements Traceability Matrix

Table 30. Requirements traceability matrix

Req.	Test cases	Status	Req.	Test cases	Status
UI-01	TC-06	Verified	FR-21	TC-20	Verified
UI-02	TC-14	Verified	FR-22	TC-21	Verified
HW-01	TC-10	Verified	FR-23	TC-22	Verified
SW-01	TC-01	Verified	FR-24	TC-23	Verified
SW-02	TC-08	Verified	NFR-01	TC-10	Verified
SW-03	TC-23	Verified	NFR-02	TC-02	Verified
COM-01	TC-11	Verified	NFR-03	TC-13	Verified
FR-01	TC-01	Verified	NFR-04	TC-06	Verified
FR-02	TC-02	Verified	NFR-05	TC-12	Verified
FR-03	TC-03	Verified	NFR-06	TC-10	Verified
FR-04	TC-03	Verified	NFR-07	TC-09, TC-25	Verified
FR-05	TC-04	Verified	NFR-08	TC-12, TC-15	Verified
FR-06	TC-05	Verified	NFR-09	TC-11	Verified
FR-07	TC-06	Verified	NFR-10	TC-02	Verified
FR-08	TC-06	Verified	NFR-11	TC-24	Verified
FR-09	TC-08	Verified	NFR-12	TC-23	Verified
FR-10	TC-04, TC-07	Verified	NFR-13	TC-26	Verified
FR-11	TC-05	Verified	SO-01	TC-11	Verified
FR-12	TC-06	Verified	SO-02	TC-09, TC-25	Verified
FR-13	TC-12	Verified	SO-03	TC-02	Verified
FR-14	TC-09	Verified	SR-01	TC-11	Verified
FR-15	TC-14	Verified	SR-02	TC-11	Verified
FR-16	TC-15	Verified	SR-03	TC-09	Verified

Req.	Test cases	Status	Req.	Test cases	Status
FR-17	TC-16	Verified	SR-04	TC-11	Verified
FR-18	TC-17	Verified	SR-05	TC-02, TC-15	Verified
FR-19	TC-18	Verified	SR-06	TC-23	Verified
FR-20	TC-19	Verified	SR-07	TC-20	Verified

18. Test Execution Summary

Table 31. Execution summary (test run of 2026-10-30)

Measure	Target	Result
Test cases passed	All	26 of 26
Automated tests passed	All	152 of 152
Requirements verified	All	54 of 54
Combined statement and branch coverage	>= 85%	87.2%
Statement coverage (application modules)	Reported	89.1%
Branch coverage (application modules)	Reported	79.4%
pylint score	>= 8.5	9.82
Training time (NFR-01)	< 300 s	15.3 s
Prediction latency, median (NFR-11)	< 200 ms	11.3 ms

Production model on the held-out partition: accuracy 75.66%, precision 52.92%, recall 75.13%, F1 0.621, ROC-AUC 0.841.

Table 32. Paradigm benchmark results

Paradigm	Model	Accuracy	Recall	ROC-AUC	Parameters
ML	XGBoost	75.66%	79.41%	0.846	5,628
DML	Deep MLP	73.24%	79.68%	0.841	5,249
ML	Logistic Regression	73.67%	79.41%	0.839	20
QML	Quantum VQC	73.74%	75.40%	0.829	61

Table 33. Defects found and resolved

ID	Defect	Found by	Resolution	Status
DEF-01	app.py imported altair, which no requirements file declared	TC-10	Declared altair in requirements.txt	Closed
DEF-02	Process could hang on macOS when PyTorch ran multi-threaded after XGBoost	Benchmark run	PyTorch inference runs single-threaded	Closed
DEF-03	XGBoost model failed to load when PyTorch was imported first (macOS OpenMP)	TC-21 run	XGBoost imported before PyTorch in dml_model.py	Closed
DEF-04	Training exported the last trained model instead of the best one	Code inspection	Champion selected by model.champion_metric	Closed
DEF-05	Early stopping could restore an epoch other than the best one	TC-21	Exact minimum validation loss tracked	Closed

Conclusion: all 26 test cases passed and all 54 requirements are verified. CCPS release 2.0 meets the acceptance criteria of this plan.